

RAMESH REDDY CHANGAL

rameshreddychangal@gmail.com — (667) 260-3005 — Chicago, IL — github.com/rameshdragon — linkedin.com/in/ramesh-reddy-changal

Education

University of Maryland, Baltimore County

Aug 2023 – May 2025

Master of Science, Computer Science GPA: 3.6/4.0

Baltimore, MD

- Coursework: Algorithms & Data Structures · Distributed & Parallel Systems · Large-Scale System Design · Machine Learning · Natural Language Processing · Information Retrieval · Computer Networks · Software Engineering · Database Systems

Skills

Languages: Python, Java, C++, JavaScript, TypeScript, Go, SQL

Frameworks: React, Next.js, Node.js, Spring Boot, FastAPI, Flask, TensorFlow, PyTorch, Scikit-learn, LangChain, gRPC

Cloud & Infra: GCP (Cloud Run, GCS, BigQuery) · AWS (EC2, S3, Lambda, ECS, RDS, SQS) · Docker · Kubernetes · Terraform · CI/CD · GitHub Actions

Systems: Distributed & Parallel Systems · Large-Scale System Design · Concurrency & Multi-Threading · Microservices · Kafka · Redis · PostgreSQL · MongoDB · REST APIs · Unix/Linux · Security Best Practices

Experience

Graduate Teaching Assistant, University of Maryland, Baltimore County

Aug 2024 – May 2025

- Reduced assignment error rate by 30% and improved average scores by 20% across 150+ students by identifying 3 recurring conceptual gaps and redesigning course rubrics to fix root causes upstream; updated rubrics carried into the following semester
- Evaluated correctness and time/space complexity of algorithmic implementations weekly; conducted structured code reviews and mentored students in distributed systems, NLP, and algorithm design
- Collaborated with faculty to redesign 3 problem sets per semester, aligning coursework with industry-standard algorithm design and formal computational theory

Software Engineering Intern, Path Creators

Jun 2021 – Jul 2021

- Reduced API latency by 40% (320ms to 190ms) replacing sequential DB queries with O(1) hash map lookup across 5 REST endpoints; wrote well-tested code with unit and integration tests, applied security best practices throughout (Python, PostgreSQL, Unix/Linux)
- Built NLP chatbot for IoT sensor recommendations using RASA end-to-end – React, Node.js, AWS – 500+ users in first month; designed multi-threaded request handling for concurrent user sessions
- Isolated race condition in Jest async mock resolution killing 15% of CI runs; two days to find, one line to fix; documented and shared with team, issue never recurred across 3 subsequent releases

Research Intern, ICRISAT

Sep 2022 – Oct 2022

- Redesigning distributed data ingestion pipeline from $O(n^2)$ dedup to $O(n \log n)$ sort-merge – 40% reliability gain across 50,000+ daily records; applied concurrency patterns to parallelize data transformation stages (Python, Pandas, Unix/Linux)
- Evaluated Decision Tree, Random Forest, and SVM across 12 metrics; deployed best model at 87% accuracy via Flask REST API; built offline-capable DHIS-2 data forms eliminating 25% session data loss for 200+ field researchers

Projects

Distributed File System (*GFS-Inspired*)

Python · gRPC · Docker · AWS — github.com/rameshdragon/distributed-fs

- Distributed and parallel storage with 3x chunk replication, consistent hashing, Raft leader election, multi-threaded heartbeat failure detection in 8s; B-tree indexing cut metadata lookup by 70%; 99.9% availability confirmed under sustained node kill tests
- Implemented concurrent chunk transfer with thread-safe write locking; gRPC for high-performance inter-node communication; security hardened with input validation and authenticated node registration

Real-Time Messaging Platform

Node.js · Kafka · Redis · React · PostgreSQL · GCP — github.com/rameshdragon/realtime-chat

- Scaled to 10,000+ concurrent users with partitioned Kafka topics for parallel message processing and Redis O(1) presence lookups; under 50ms p99 delivery; deployed on GCP Cloud Run at 99.9% uptime

AI Icon Generation SaaS

React · Node.js · TypeScript · DALL-E 3 · AWS Lambda · Stripe —

github.com/rameshdragon/ai-icon-generator

- Developed and shipped full-stack web application solo – React/TypeScript, Node.js, DALL-E 3, Stripe payments, AWS serverless; rate limiting, security middleware, 5 generation modes, CI/CD pipeline; real users, real payments

ML Crop Yield Prediction

Python · Scikit-learn · TensorFlow · Flask · React · GCP — github.com/rameshdragon/crop-yield

- Compared Random Forest, Decision Tree, Gradient Boosting on 2,000 samples; 23% accuracy gain over baseline; 80% preprocessing speedup replacing loops with vectorized NumPy; deployed on GCP with React dashboard for non-technical users